GAME PROGRAMMING COURSE — UNITY

Inspector とプロジェクトの整理

TIDY UP

属性で Inspector を使いやすくし、フォルダ・Prefab・シーンを片付ける

対象：1年生 ｜ 2コマ

PART 1 | 1コマ目

Inspector を使いやすくする属性

1

[ ] をつけるだけで、Inspector が見やすく・ミスしにくくなる

属性とは

[ ] で変数やクラスに「目印」をつける

```
[SerializeField] float moveSpeed = 5f;
```

[SerializeField] の部分 ＝ 属性

Unity への目印。英語では Attribute（アトリビュート）。

残りの部分 ＝ いつもの変数の宣言

属性をつけても、変数そのものの使い方は変わらない。

属性 01 – 05

見た目を整える

見出し・説明・隙間・複数行

属性 06 – 09

入力ミスを防ぐ

範囲・最小値・必要なコンポーネント

属性 10 – 15

表示・非表示と便利機能

出す・隠す・まとめる・テスト用

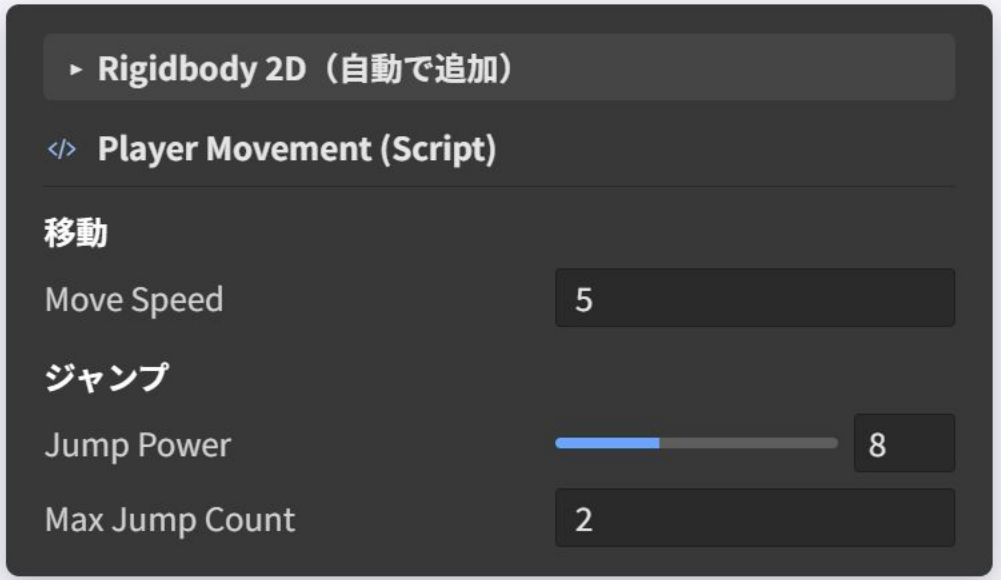
全文

属性を使ったスクリプトと、Inspector の見え方

C# PlayerMovement.cs

```
1 using UnityEngine;
2
3 [RequireComponent(typeof(Rigidbody2D))]
4 public class PlayerMovement : MonoBehaviour
5 {
6     [Header("移動")]
7     [Tooltip("1秒に進む距離")]
8     [SerializeField, Min(0f)] float moveSpeed = 5f;
9
10    [Header("ジャンプ")]
11    [SerializeField, Range(1f, 20f)] float jumpPower = 8f;
12    [SerializeField, Min(0)] int maxJumpCount = 2;
13 }
```

Inspector での見え方



POINT 属性は1行に1つずつ書いても、[SerializeField, Min(0f)] のようにカンマで並べてもよい。

属性 01 ｜ 見た目を整える

[Header] で見出しをつける

C# PlayerMovement.cs

```
1 [Header("移動")]
2 [SerializeField] float moveSpeed = 5f;
3 [SerializeField] float dashSpeed = 10f;
4
5 [Header("ジャンプ")]
6 [SerializeField] float jumpPower = 8f;
```

Inspector での見え方



できること

Inspector に太字の見出しを出して、変数をグループに分ける。見出しの下に来る変数の上に書く。

使いどころ

変数が増えてきたら、移動・ジャンプ・攻撃など役割ごとに分けると探しやすい。

属性 02 ｜ 見た目を整える

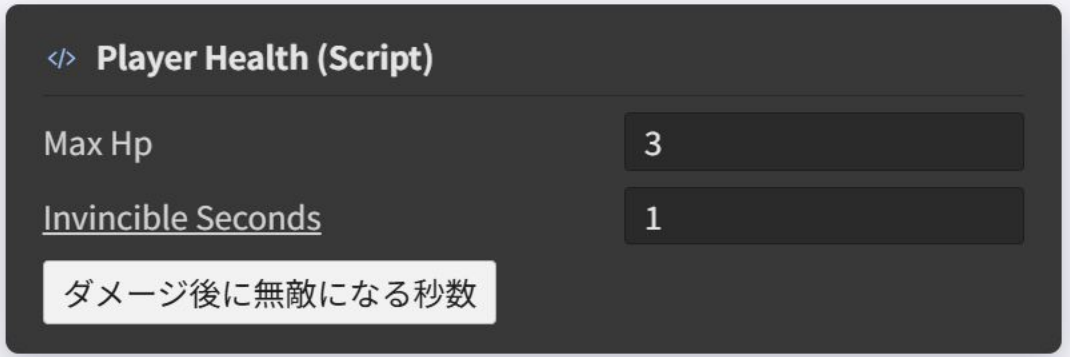
[Tooltip] で説明をつける

C# PlayerHealth.cs

```cs
[Tooltip("最大HP。0 にするとすぐ倒れる")]
[SerializeField] int maxHp = 3;

[Tooltip("ダメージ後に無敵になる秒数")]
[SerializeField] float invincibleSeconds = 1f;
```

Inspector での見え方（マウスを乗せたとき）



できること

変数名にマウスを乗せると説明が出る。単位や「何を入れるか」「入れるとどうなるか」を書ける。

使いどころ

名前だけでは単位や意味が伝わらない値。チームの仲間が調整する値には特に書いておく。

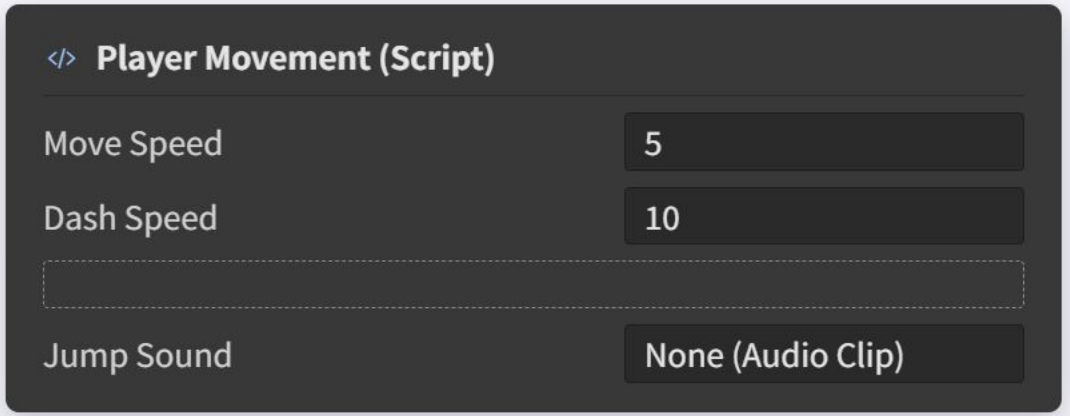
属性 03 ｜ 見た目を整える

[Space] で隙間を空ける

C# PlayerMovement.cs

```
1 [SerializeField] float moveSpeed = 5f;
2 [SerializeField] float dashSpeed = 10f;
3 [Space(20)]
4 [SerializeField] AudioClip jumpSound;
```

Inspector での見え方（点線が隙間）



できること

変数と変数のあいだに隙間を空ける。( ) の数字で隙間の大きさを指定できる。

使いどころ

見出しをつけるほどではないが、少し区切りたいとき。数値と音・画像の間など。

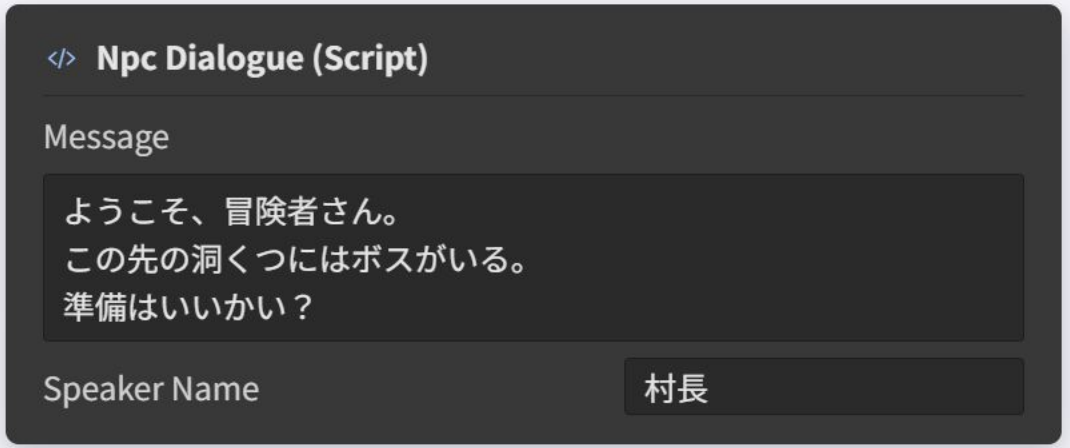
属性 04 ｜ 見た目を整える

[TextArea] で長い文字を入力しやすくする

C# NpcDialogue.cs

```
1 // 最低3行、最大5行の入力欄
2 [TextArea(3, 5)]
3 [SerializeField] string message;
4
5 [SerializeField] string speakerName;
```

Inspector での見え方



できること

string を複数行の大きな入力欄にする。改行もそのまま入れられる。普通の string は1行だけ。

使いどころ

NPC のセリフ、アイテムの説明文、看板の文章など、長い文字列を Inspector で書きたいとき。

属性 05 ｜ 見た目を整える

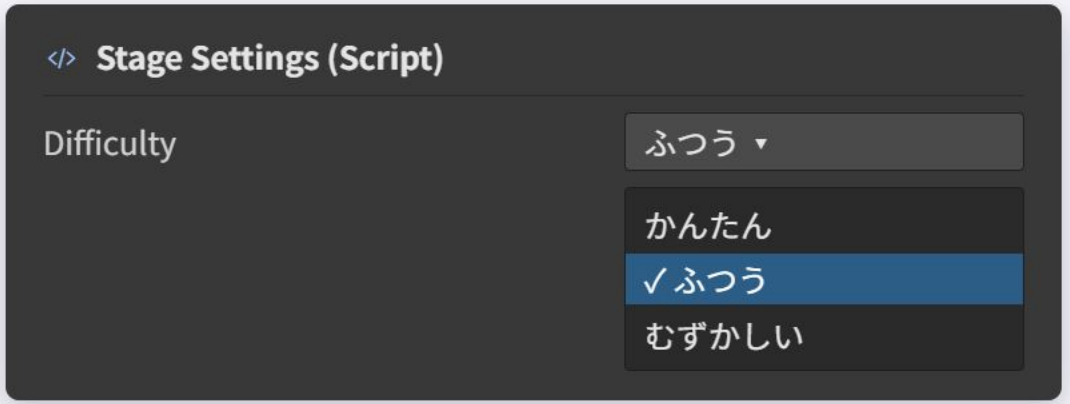
[InspectorName] で enum の表示名を変える

C# StageSettings.cs

```cs
public enum Difficulty
{
    [InspectorName("かんたん")] Easy,
    [InspectorName("ふつう")] Normal,
    [InspectorName("むずかしい")] Hard
}

[SerializeField] Difficulty difficulty;
```

Inspector での見え方（ドロップダウンを開いたとき）



できること

enum の値を、Inspector のドロップダウンでは別の名前で表示する。コードの名前は英語のまま。

使いどころ

難易度・敵の種類・アイテムの種類など、プログラマー以外も Inspector で選ぶ項目。

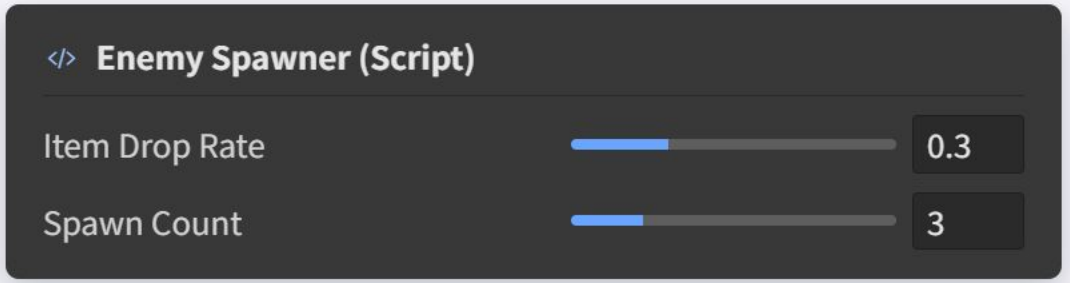
属性 06 ｜ 入力ミスを防ぐ

[Range] でスライダーにする

C# EnemySpawner.cs

```
1 [Range(0f, 1f)]
2 [SerializeField] float itemDropRate = 0.3f;
3
4 [Range(1, 10)]
5 [SerializeField] int spawnCount = 3;
```

Inspector での見え方



できること

スライダーで値を調整できる。範囲外の値は Inspector から入れられない。int にも使える。

使いどころ

確率・音量・割合など「この範囲でしか意味がない」値。プレイしながら調整したい値。

属性 07 ｜ 入力ミスを防ぐ

[Min] でマイナスを入れられなくする

C# PlayerHealth.cs

```
1 [Min(1)]
2 [SerializeField] int maxHp = 3;
3
4 [Min(0f)]
5 [SerializeField] float invincibleSeconds = 1f;
```

Inspector での見え方（−3 を入力したとき）



できること

最小値を決める。それより小さい値を入れると、自動で最小値に直る。上限はつかない。

使いどころ

HP・速さ・個数・秒数など、マイナスになると困る値。上限も決めたいなら [Range]。

属性 08 ｜ 入力ミスを防ぐ

[RequireComponent] で必要な部品を自動で付ける

C# PlayerMovement.cs

```
1 [RequireComponent(typeof(Rigidbody2D))]
2 public class PlayerMovement : MonoBehaviour
3 {
4     Rigidbody2D body;
5
6     void Awake()
7     {
8         body = GetComponent<Rigidbody2D>();
9     }
10 }
```

Inspector での見え方（スクリプトを付けたとき）



できること

スクリプトを付けると、必要なコンポーネントも自動で付く。必要なものを外そうとすると止めてくれる。

使いどころ

GetComponent で取るコンポーネントがあるスクリプト。付け忘れによる null エラーを防げる。

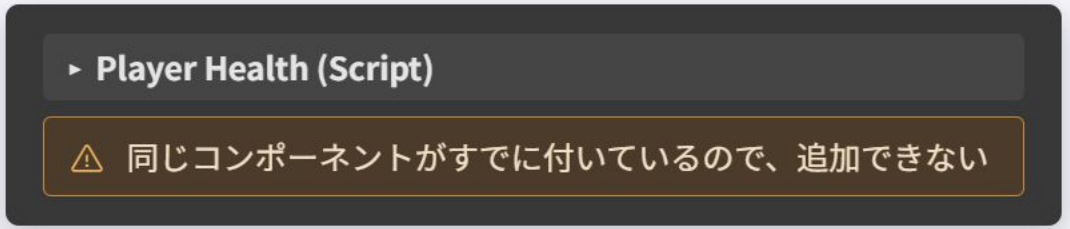
属性 09 ｜ 入力ミスを防ぐ

[DisallowMultipleComponent] で二重付けを防ぐ

C# PlayerHealth.cs

```cs
[DisallowMultipleComponent]
public class PlayerHealth : MonoBehaviour
{
    [SerializeField, Min(1)] int maxHp = 3;
}
```

2つ目を付けようとしたとき



できること

同じスクリプトを1つの GameObject に2つ付けられなくする。

使いどころ

HP・移動・入力など、2つ付くと「ダメージが2回入る」「2倍速く動く」バグになるスクリプト。

属性 11 ｜ 表示・非表示

[HideInInspector] で public を隠す

C# ScoreCounter.cs

```
1 // 他のスクリプトからは読みたいが、Inspector には出さない
2 [HideInInspector] public int currentScore;
3
4 [SerializeField] int bonusScore = 100;
```

Inspector での見え方



できること

public の変数を Inspector に出さないようにする。他のスクリプトからは今までどおり使える。

使いどころ

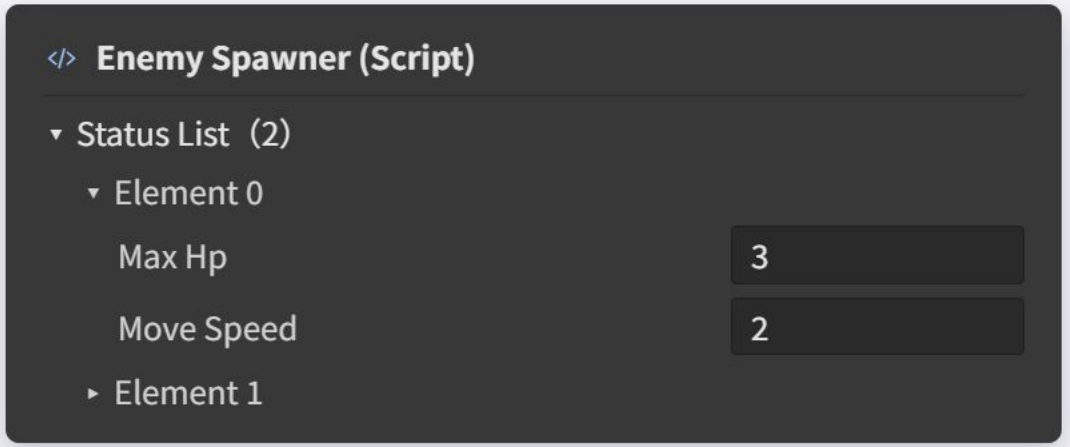
ゲーム中に変わる値で、Inspector でいじられたくないもの。ただし多くの場合は private で足りる。

[System.Serializable] で自作の型を表示する

C# EnemySpawner.cs

```
1 [System.Serializable]
2 public class EnemyStatus
3 {
4     public int maxHp = 3;
5     public float moveSpeed = 2f;
6 }
7
8 // EnemySpawner クラスの中
9 [SerializeField] EnemyStatus[] statusList;
```

Inspector での見え方（配列にした場合）



できること

自分で作ったクラスや構造体を、折りたたみ表示で編集できる。付けないと Inspector に出ない。

使いどころ

敵のステータスや武器の性能など、まとめて扱う値のセット。配列にすると一覧で編集できる。

属性 15 ｜ 便利機能（発展）

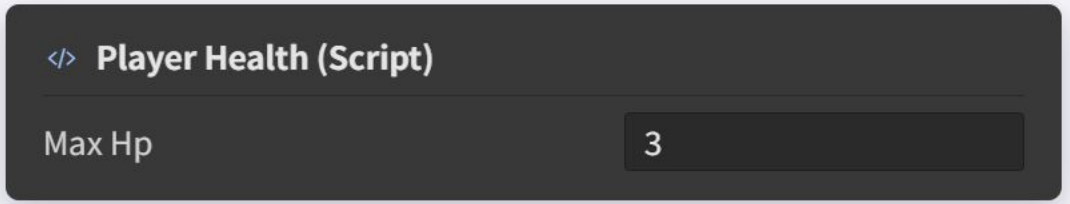
[field: SerializeField] でプロパティを表示する

C# PlayerHealth.cs

```cs
[field: SerializeField]
public int MaxHp { get; private set; } = 3;

// 別のスクリプトから
int hp = playerHealth.MaxHp;   // 読める
playerHealth.MaxHp = 999;     // エラー
```

Inspector での見え方



できること

プロパティを Inspector に表示する。外からは読めるが、書きかえはこのクラスの中だけ。

使いどころ

「Inspector で設定したい・他から読みたい・でも書きかえられたくない」値。プロパティを習ってから。

PART 2 ｜ 2コマ目

Unity プロジェクトの整理

2

フォルダ・Prefab・シーン・ファイルを、迷わない形にしておく

整理 01 ｜ フォルダ

基本のフォルダ構成

Project ウィンドウ

```
▾ Assets
    ▸ Animations
    ▸ Audio
    ▸ Materials
    ▸ Prefabs
    ▸ Scenes
    ▸ Scripts
    ▸ Sprites
```

フォルダ	入れるもの
Scenes	シーン（.unity）
Scripts	C# スクリプト（.cs）
Prefabs	Prefab
Sprites	画像（.png など）
Audio	効果音・BGM
Animations	アニメーション・Animator Controller
Materials	マテリアル（2D ではあまり使わない）

ポイント

迷ったら「ファイルの種類」で分ける。Assets の直下にはファイルを直接置かない。

注意

作り始めに用意しておく。あとからまとめて移動すると、何がどこにあるか分からなくなる。

整理 02 ｜ フォルダ

種類別と機能別、2つの分け方

種類別

```
▾ Assets
    ▾ Scripts
        ▸ Player
        ▸ Enemy
    ▾ Sprites
        ▸ Player
        ▸ Enemy
```

スクリプトだけ見たい、画像だけ探したいときに便利。Unity の入門ではこちらが多い。

機能別

```
▾ Assets
    ▾ Player
        ▸ Scripts
        ▸ Sprites
    ▾ Enemy
        ▸ Scripts
        ▸ Sprites
```

敵1種類に関するものが1か所にまとまる。作品が大きく、担当が分かれるときに便利。

ポイント

1年生の作品なら種類別で十分。ファイルが増えたら、種類別の中を機能で分ける。

注意

チームでは最初にどちらにするか決める。途中で2つが混ざるのが一番探しにくい。

特別な意味を持つフォルダ名

フォルダ名	何が起きるか	注意
Resources	Resources.Load で名前から読み込める。中身は使っていなくても全部ビルドに入る	素材置き場にしない。ゲームが重くなる
Editor	中のスクリプトはエディタ専用で、ビルドしたゲームには入らない	ゲーム用のスクリプトを入れるとビルドで動かない
StreamingAssets	中のファイルが、そのままの形でビルドにコピーされる	動画や設定ファイル用。Assets の直下に置く
Plugins	外部のライブラリ（DLL など）を置く場所として使われる	普段は自分で作らない
Gizmos	Scene ビューに表示するアイコン画像を置く	1年生はほぼ使わない

POINT 知らずにこの名前のフォルダを作らない。Resources と Editor は、どの階層にあっても特別扱いになる。

整理 04 ｜ フォルダ

自作のものと外部の素材を分ける

Project ウィンドウ（例）

```
▾ Assets
    ▾ MyGame   ← 自作はここ
        ▸ Prefabs
        ▸ Scenes
        ▸ Scripts
        ▸ Sprites
    ▸ ForestTileset   ← ストアの素材
    ▸ TextMesh Pro   ← Unity が追加
```

1. どれを消してよいか、すぐ分かる
2. 提出・公開してよい素材か（利用規約）を確認しやすい
3. 素材を入れ替えるとき、自作のものを巻きこまない

ポイント

自分で作ったものは1つのフォルダにまとめる。外部の素材はインポートした場所のままにしておく。

注意

外部素材は利用規約を必ず確認する。クレジット表記が必要なものや、再配布できないものがある。

消してもよいフォルダ、消してはいけないフォルダ

プロジェクトの中	提出・コピー	中身
Assets	必要	自分で作ったもの・素材のすべて
Packages	必要	使っているパッケージの一覧
ProjectSettings	必要	タグ・レイヤー・入力などの設定
UserSettings	なくてもよい	自分だけのエディタの設定
Library	除いてよい	読み込み用のキャッシュ。開くと作り直される
Temp ・ Logs ・ obj	除いてよい	作業用の一時ファイル・記録
.sln ・ .csproj	除いてよい	Visual Studio 用。Unity が自動で作る

ポイント

提出や USB での持ち運びは、Assets・Packages・ProjectSettings の3つがあれば開ける。サイズが大きく減る。

注意

Library を除くと、次に開くときの読み込みに時間がかかる。Unity で開いている最中には消さない。

整理 15 ｜ ファイル

使っていないものを片付ける

- 試しに作ったシーン（SampleScene・Test など）
- 使っていないスクリプト（NewBehaviourScript など）
- 差しかえ前の画像・使わなくなった音
- 試しにインポートしたが使わなかった素材

消す前の手順

1. バックアップを取る（コピーや Git）
2. 右クリック → Find References In Scene で使われていないか確認
3. Project ウィンドウで削除し、ゲームを動かして確認

POINT いらないファイルが多いほど、必要なファイルを探す時間が増える。作品の区切りごとに片付ける。